Problem 1

Before we can calculate E_{k-fold}, we must be able to calculate $E_{hold-out}$ which is define on page 47 of the textbook as $\mathscr{T}$

$$E_{hold-out} := \frac{1}{n_v}\sum_{j=1}^{n_v} E(\hat{y}((\boldsymbol{x}_j^{'}; \mathscr{T}), y_j^{'})$$

In [1]:
```
def calculate_squared_error(y_hat_j, y_j):
    return (y_hat_j - y_j)**2

def calculate_holdout_error(y_hat, y, error_func=calculate_squared_error)
    n_v = y_hat.shape[0]
    total_squared_error = 0
    for y_hat_j, y_j in zip(y_hat, y):
        total_squared_error += error_func(y_hat_j, y_j)

    return (1/n_v)*total_squared_error
```

I refer to page 70 of the textbookd for the definition of E_{k-fold}

$$E_{k-fold} := \frac{1}{k}\sum_{l=1}^{k} E_{hold-out}^{(l)}$$

In [2]:
```
import random
import numpy as np
from copy import deepcopy

def _get_batches(X, y, k):
    batches = []
    for i in range(k):
        batches.append(([], []))
    randomized_indices = random.sample([i for i in range(y.shape[0])], y.

    i = 0
    for j in randomized_indices:
        if i >= k:
            i = 0
        batches[i][0].append(X[j, :])
        batches[i][1].append(y[j])

        i += 1

    for i in range(k):
        X_i, y_i = batches[i]
        batches[i] = (np.array(X_i), np.array(y_i))

    return batches


def calculate_k_fold_error(regressor, X, y, k):
    batches = _get_batches(X, y, k)
```

```
    holdout_error = []
    for i, (X_l, y_l) in enumerate(batches):
        # training_batches = [batches[j] for j in range(len(batches)) if
        X_trains = [batches[j][0] for j in range(k) if not j == i]
        y_trains = [batches[j][1] for j in range(k) if not j == i]
        X_train = np.concatenate(X_trains)
        y_train = np.concatenate(y_trains)


        _regressor = deepcopy(regressor)
        _regressor.fit(X_train, y_train)
        y_hat_l = _regressor.predict(X_l)
        holdout_error.append(calculate_holdout_error(y_hat_l, y_l))

    return (1/k)*np.sum(holdout_error)
```

In [3]:
```
from sklearn.linear_model import LinearRegression
import numpy as np

X = np.array([[168], [168], [250], [192], [252], [170], [217], [157], [21
y = np.array([40, 33, 32, 25, 24, 36, 35, 35, 26, 23, 36, 29, 27, 26, 22,

regressor = LinearRegression()
print("E-kfold (k=30): ", calculate_k_fold_error(regressor, X, y, 30))
print("E-kfold (k=3): ", calculate_k_fold_error(regressor, X, y, 3))
```

```
E-kfold (k=30):  19.103971146863316
E-kfold (k=3):  19.244300388337386
```

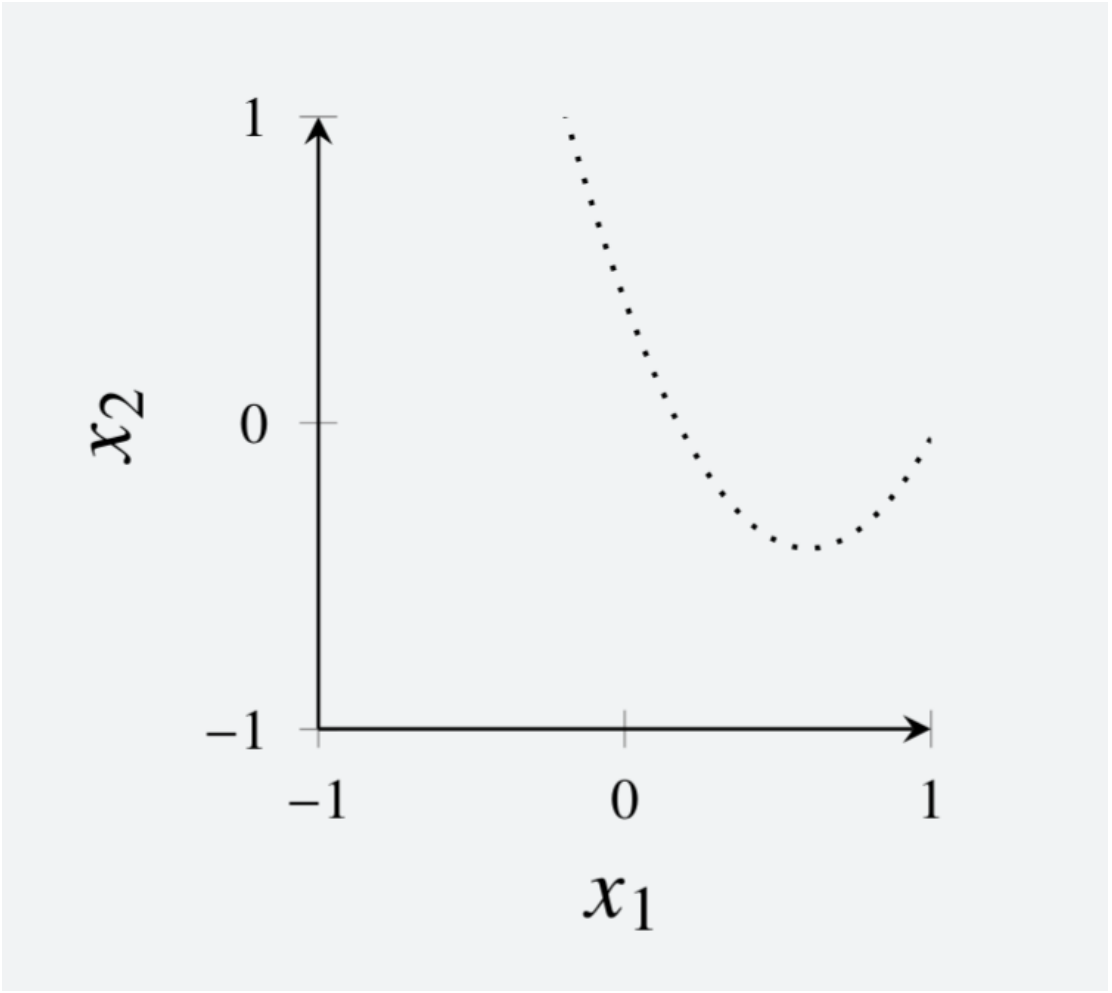
The case where k=30 is known as the leave-one-out cross-validation case because the metaparameter k is equal to n, the number of training data points. Thus, we train the model on 29 (n-1) data points, leaving one out for validation. Then we average these 29 data points. Since the leave one out cross validation procedure always results in averaging the same n-1 numbers (though the order is different) the value of E-kfold will be the same regardless of which student wrote the code. Therefore, the k=30 case will be the same from student to student.

The k=3 case will result in a different value from student to student (and even upon different runs to the same code depending on whether the randomizer is deterministic or not). This is because the set of batches of 27 and the holdout batches are not guaranteed to be the same from run to run.

Additionally, the k=30 case is likely more accurate, because the number of training points being used to train (29) in the models used to estimate E-kfold, is much closer to the number of training points in the final model (30).

Problem 2 - Recreating Ex. 4.1

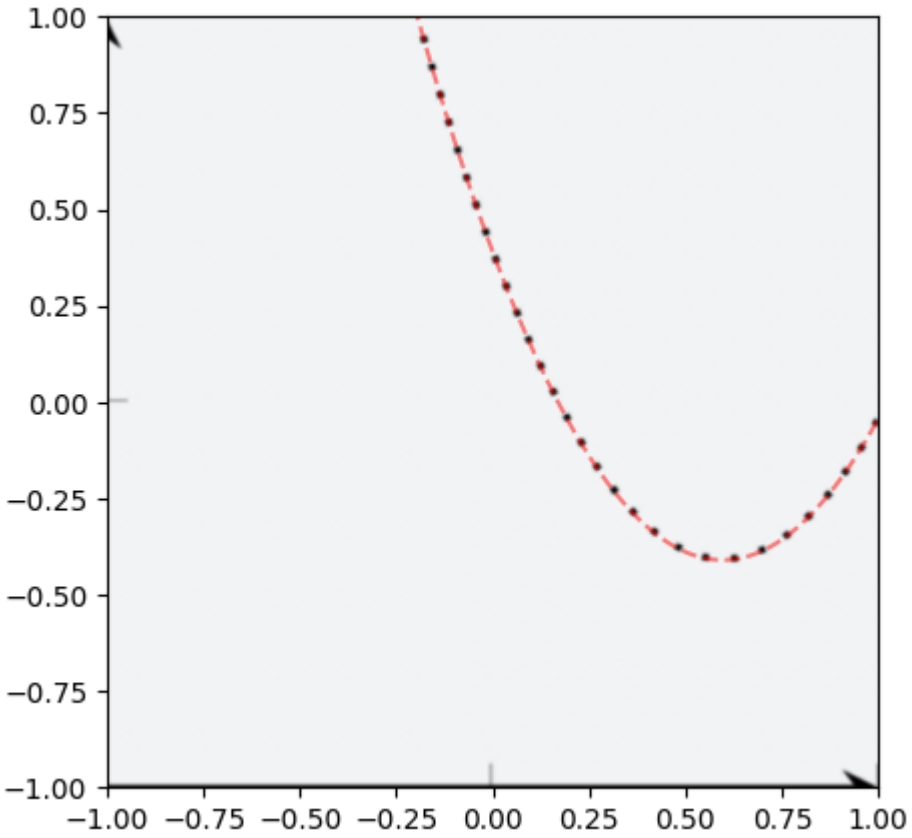
All points above the plotted line are blue with probability of 0.8, and all points below the plotted line are red with probability of 0.8.



In [4]:
```
import matplotlib.pyplot as plt
BLUE = -1
RED = 1

img = plt.imread("cropped_curve.png")
fig, ax = plt.subplots(figsize=(5, 5))
ax.imshow(img, extent=[-1, 1, -1, 1])

def curve(x):
    return 2.247*(x-0.5965)**2 - 0.41

@np.vectorize
def generate_data_point(x, y):
    data_point = None
    if y > curve(x):
        return np.random.choice([BLUE, RED], size=1, p=[0.8, 0.2])[0]
    elif y <= curve(x):
        return np.random.choice([BLUE, RED], size=1, p=[0.2, 0.8])[0]

def generate_data_points(n):
    x1 = np.linspace(-1, 1, 50)
    x2 = np.linspace(-1, 1, 50)
    X1, X2 = np.meshgrid(x1, x2)

    random_x1_indices = random.sample([i for i in range(X1.ravel().shape[
    random_x2_indices = random.sample([i for i in range(X2.ravel().shape[

    random_x1_values = X1.ravel()[random_x1_indices]
    random_x2_values = X2.ravel()[random_x2_indices]
```

```
    X = np.array([[x1, x2] for x1, x2 in zip(random_x1_values, random_x2_
    y = np.array([generate_data_point(x1, x2) for x1, x2 in zip(random_x1
    return X, y

x = np.linspace(-0.5, 1.0, 1000)
plt.plot(x, curve(x), "r--", alpha=0.5)
plt.xlim([-1, 1])
plt.ylim([-1, 1])
```

Out[4]: (-1.0, 1.0)

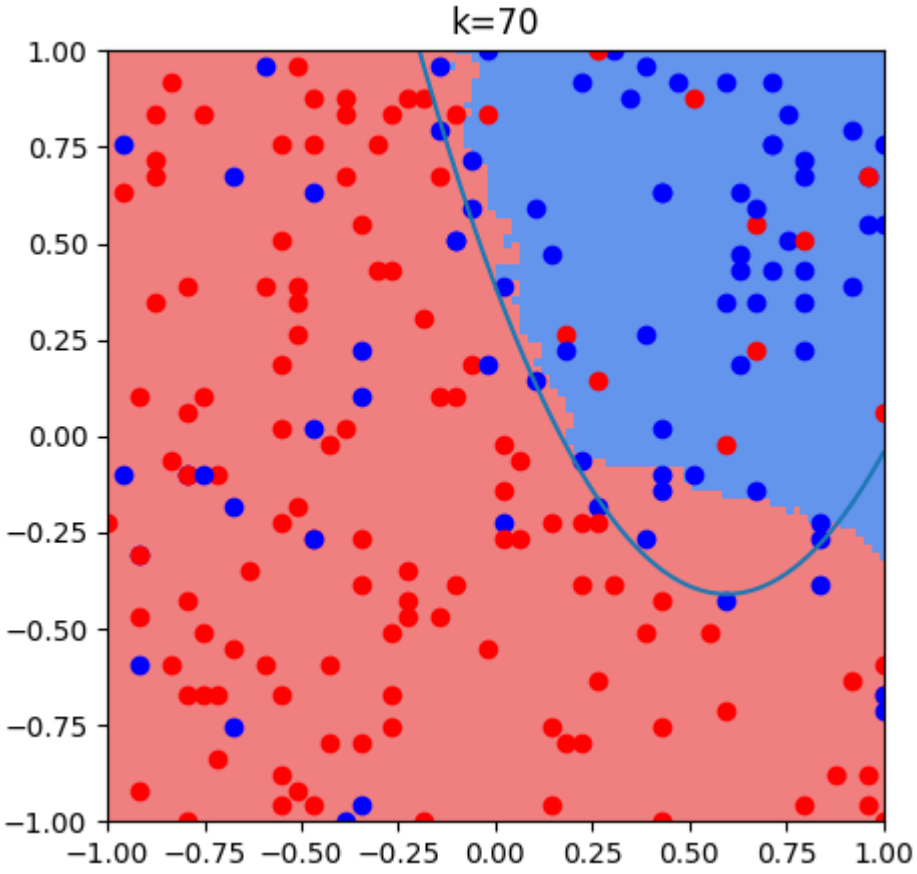


In [5]:
```
from sklearn.neighbors import KNeighborsClassifier
from matplotlib.colorizer import Colorizer
from matplotlib.colors import ListedColormap

def scatter_points(ax, X, y):
    for x, _y in zip(X, y):
        if _y == RED:
            color = "red"
            marker = '1'
        elif _y == BLUE:
            color = "blue"
            marker = '2'
        # ax.scatter(x[0], x[1], color=color, marker=marker)
        ax.scatter(x[0], x[1], color=color)

def plot_decision_regions(ax, classifier, xlim=[-1, 1], ylim=[-1, 1]):
    colorizer=Colorizer(ListedColormap(['cornflowerblue', 'lightcoral']))

    x1 = np.linspace(xlim[0], xlim[1], 100)
    x2 = np.linspace(ylim[0], ylim[1], 100)

    @np.vectorize
    def calculate_z(x1, x2):
```

```
        return classifier.predict([[x1, x2]])[0]

    X1, X2 = np.meshgrid(x1, x2)
    Z = calculate_z(X1, X2)

    # for (x1, x2), z in zip(zip(X1.ravel(), X2.ravel()), Z.ravel()):
    #     if x1 > 0.5 and x2>0.1:
    #         pass
    ax.pcolormesh(X1, X2, Z, colorizer=colorizer)

# n = 200

# random_x = np.random.uniform(-1, 1, n)
# random_y = np.random.uniform(-1, 1, n)

# X = np.array([[x, y] for x, y in zip(random_x, random_y)])
# y = np.array([generate_data_point(x, y) for x, y in zip(random_x, rando
```

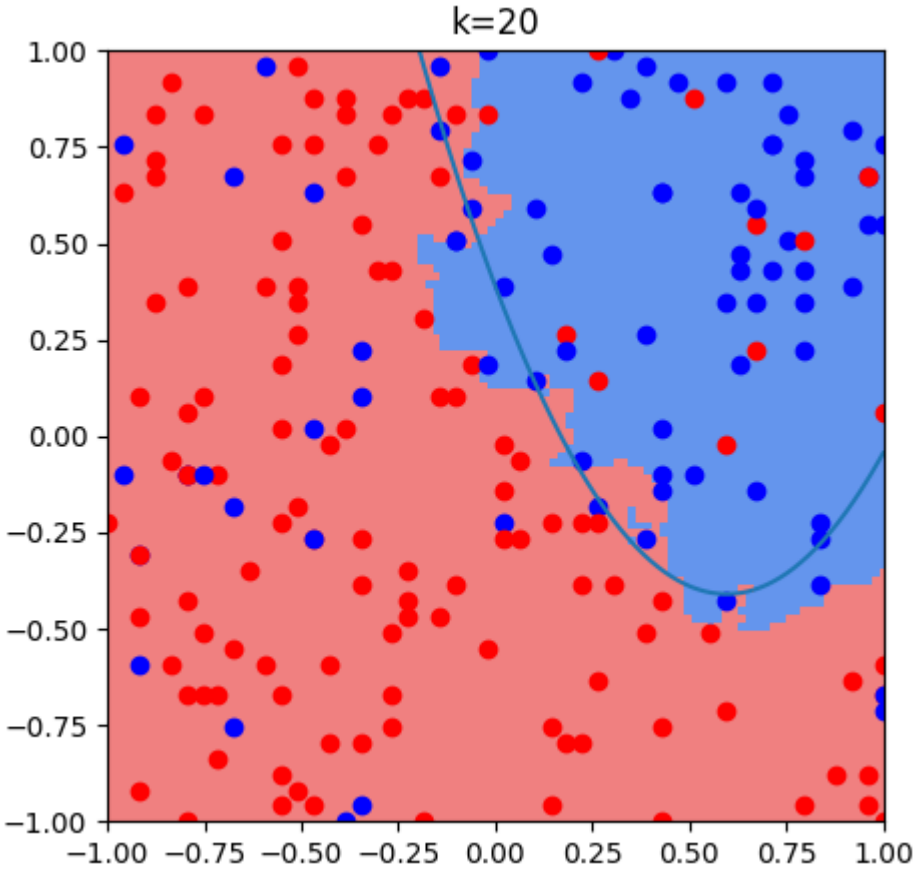
In [6]:
```
fig, ax = plt.subplots(figsize=(5, 5))

n = 200
k = 70

X, y = generate_data_points(n)
classifier_k70 = KNeighborsClassifier(n_neighbors=k, algorithm="brute").f

plot_decision_regions(ax, classifier_k70)
scatter_points(ax, X, y)

x = np.linspace(-1, 1, 1000)
ax.plot(x, curve(x))
ax.set_xlim([-1, 1])
ax.set_ylim([-1, 1])
ax.set_title(f"k={k}")
```

Out[6]:
```
Text(0.5, 1.0, 'k=70')
```



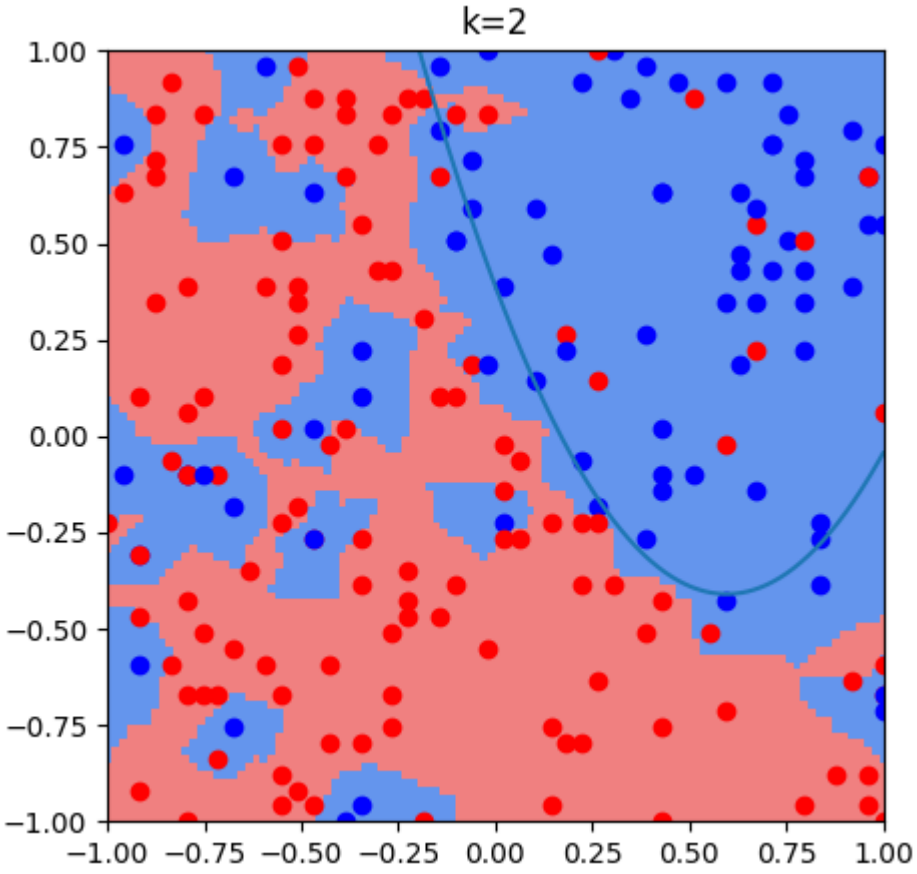
In [7]:
```
fig, ax = plt.subplots(figsize=(5, 5))

# n = 200
k = 20

# X, y = generate_data_points(n)
classifier_k20 = KNeighborsClassifier(n_neighbors=k, algorithm="brute").f

plot_decision_regions(ax, classifier_k20)
scatter_points(ax, X, y)

x = np.linspace(-1, 1, 1000)
ax.plot(x, curve(x))
ax.set_xlim([-1, 1])
ax.set_ylim([-1, 1])
ax.set_title(f"k={k}")
```

Out[7]:
```
Text(0.5, 1.0, 'k=20')
```



In [8]:
```
fig, ax = plt.subplots(figsize=(5, 5))

# n = 200
k = 2

# X, y = generate_data_points(n)
classifier_k2 = KNeighborsClassifier(n_neighbors=k, algorithm="ball_tree"

plot_decision_regions(ax, classifier_k2)
scatter_points(ax, X, y)

x = np.linspace(-1, 1, 1000)
ax.plot(x, curve(x))
ax.set_xlim([-1, 1])
ax.set_ylim([-1, 1])
ax.set_title(f"k={k}")
```

Out[8]: Text(0.5, 1.0, 'k=2')



Next we recreate the data from the table

In [9]:
```
from sklearn.metrics import mean_squared_error

def missclassification_rate(y_true, y_pred):
    n = len(y_true)

    num_errors = np.sum([0 if y_t == y_p else 1 for y_t, y_p in zip(y_tru

    return num_errors / n

classifier_error = {
    classifier_k70.n_neighbors: {"train_error": [], "new_error": []},
    classifier_k20.n_neighbors: {"train_error": [], "new_error": []},
    classifier_k2.n_neighbors: {"train_error": [], "new_error": []},
    3: {"train_error": [], "new_error": []},
}

N = 1000

for _ in range(N):
    X_train, y_train = generate_data_points(n)
    X_test, y_test = generate_data_points(n)
    for k, data in classifier_error.items():
        classifier = KNeighborsClassifier(n_neighbors=k).fit(X_train, y_t
        data["train_error"].append(missclassification_rate(y_train, class
        data["new_error"].append(missclassification_rate(y_test, classifi
```

In [10]:
```
print("E-train with k=70", np.sum(classifier_error[70]["train_error"])/N)
print("E-new   with k=70", np.sum(classifier_error[70]["new_error"])/N)

print("E-train with k=20",np.sum(classifier_error[20]["train_error"])/N)
```

```
print("E-new   with k=20",np.sum(classifier_error[20]["new_error"])/N)

print("E-train with k=2",np.sum(classifier_error[2]["train_error"])/N)
print("E-new   with k=2",np.sum(classifier_error[2]["new_error"])/N)

print("E-train with k=3",np.sum(classifier_error[3]["train_error"])/N)
print("E-new   with k=3",np.sum(classifier_error[3]["new_error"])/N)
```

```
E-train with k=70 0.23318
E-new   with k=70 0.24101499999999998
E-train with k=20 0.21793
E-new   with k=20 0.23163000000000003
E-train with k=2 0.16460999999999998
E-new   with k=2 0.36577
E-train with k=3 0.16268000000000002
E-new   with k=3 0.2776
```

Problem 3 - Recreating Ex. 4.2

After languishing over not getting the right results, I have discovered that there is a typo in the textbook. It seems that n=20 for this problem, not 10.

In [11]:
```
def simulate_data(n):
    x = random.sample(list(np.linspace(-5, 10, 10000)), n)
    X = np.array([[_x] for _x in x])

    y_mean = np.array([min(_y, 3) for _y in 0.1*np.array(x)**2])
    y = y_mean + np.random.normal(0, 1, n)
    return X, y
```

Make sure that I understand how alpha (from sklearn) and lambda (from textbook) relate. I determined that

$$\alpha = n\lambda$$

In [12]:
```
from sklearn.linear_model import Ridge
from sklearn.preprocessing import PolynomialFeatures
lam = 10.0
n = 10
X_train, y_train = simulate_data(n)
poly = PolynomialFeatures(degree=2)
X_poly_train = poly.fit_transform(X_train)
ridge = Ridge(alpha=n*lam, fit_intercept=False).fit(X_poly_train, y_train

thetas = np.linalg.inv((X_poly_train.T@X_poly_train) + n*lam*np.eye(3))@X
print(thetas)
print(ridge.coef_)

X_test, y_test = simulate_data(1)
X_poly_test = poly.fit_transform(X_test)
```

```
[-0.0241256  -0.07469416  0.04798187]
[-0.0241256  -0.07469416  0.04798187]
```

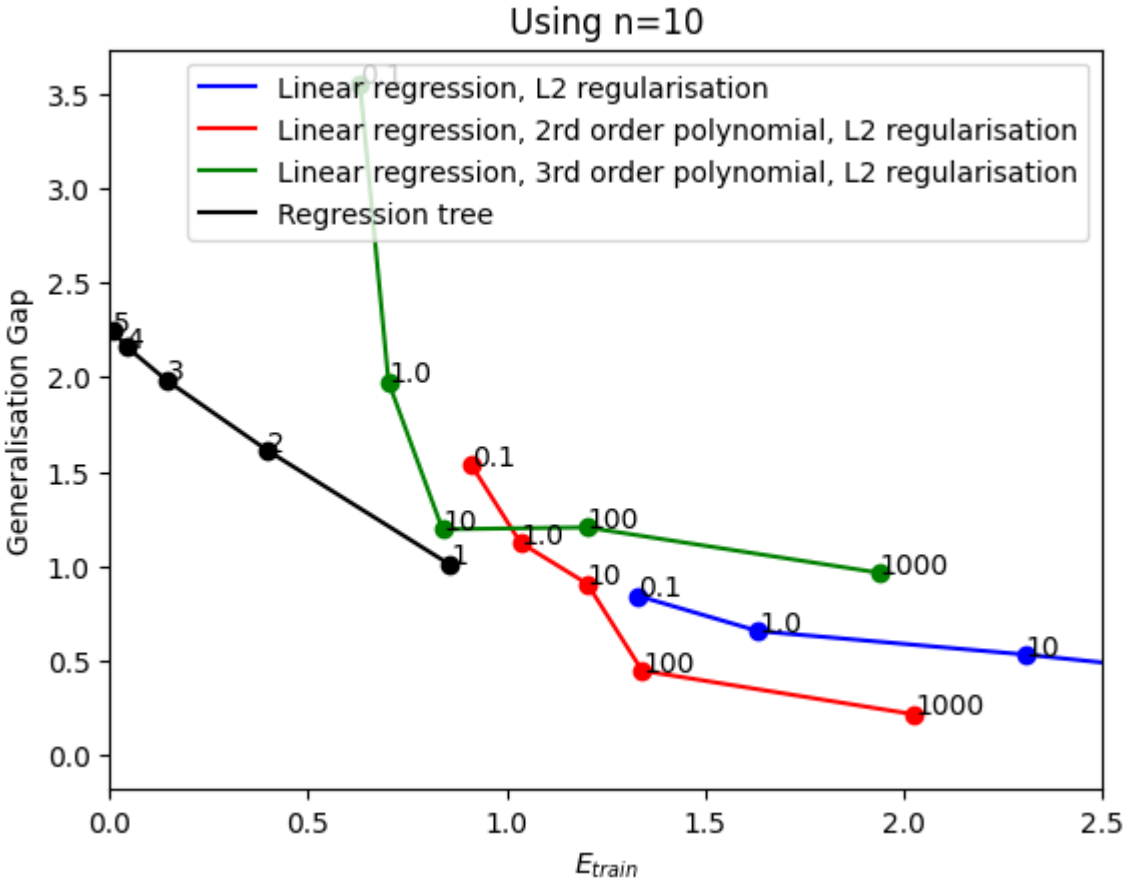
In [13]:
```
from sklearn.linear_model import Ridge
from sklearn.preprocessing import PolynomialFeatures
```

```
from sklearn.tree import DecisionTreeRegressor

def get_metrics_quadratic(n, lam, order=2):
    X_train, y_train = simulate_data(n)
    poly = PolynomialFeatures(degree=order)
    X_poly_train = poly.fit_transform(X_train)
    ridge = Ridge(alpha=n*lam, fit_intercept=False).fit(X_poly_train, y_t
    E_train = mean_squared_error(y_train, ridge.predict(X_poly_train))

    X_test, y_test = simulate_data(10*n)
    X_poly_test = poly.fit_transform(X_test)
    E_test = mean_squared_error(y_test, ridge.predict(X_poly_test))

    return E_train, E_test

def get_metrics_tree(n, tree_depth):
    X_train, y_train = simulate_data(n)
    tree = DecisionTreeRegressor(max_depth=tree_depth).fit(X_train, y_tra
    E_train = mean_squared_error(y_train, tree.predict(X_train))

    X_test, y_test = simulate_data(10*n)
    E_test = mean_squared_error(y_test, tree.predict(X_test))

    return E_train, E_test
```

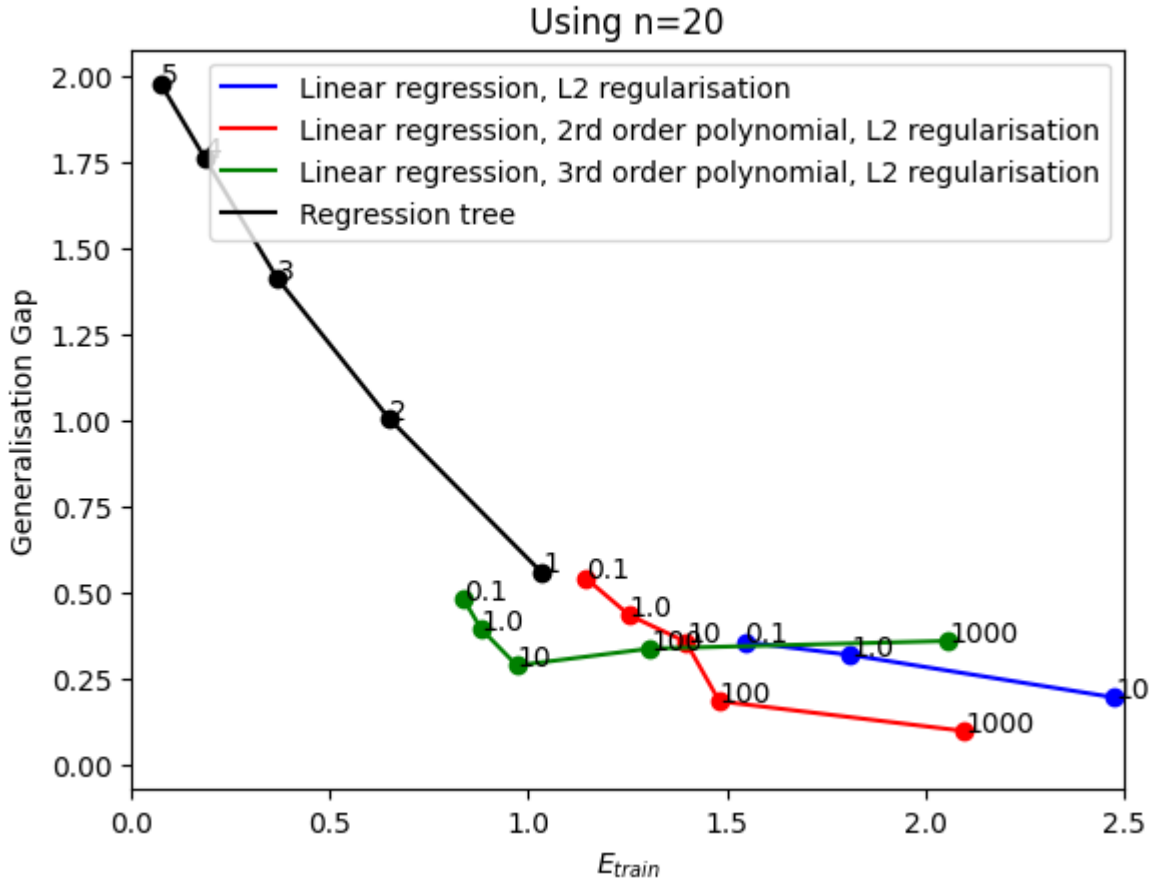
In [14]:
```
def generalisation_gap_vs_e_train(ax, get_metrics_func, meta_parameters,
    metrics = {meta_param: {"E_train": 0, "E_test": 0} for meta_param in

    for reg_param in metrics.keys():
        E_train, E_test = [], []
        for _ in range(M):
            _E_train, _E_test = get_metrics_func(n, reg_param)
            E_train.append(_E_train)
            E_test.append(_E_test)

        metrics[reg_param]["E_train"] = np.sum(np.array(E_train))/M
        metrics[reg_param]["E_test"] = np.sum(np.array(E_test))/M

    train_error = []
    generalisation_gap = []
    meta_params = []

    for meta_param, data in metrics.items():
        train_error.append(data['E_train'])
        generalisation_gap.append(data['E_test'] - data['E_train'])
        meta_params.append(meta_param)

    ax.plot(train_error, generalisation_gap, color=color, label=label)
    for meta_param, (x, y) in zip(meta_params, zip(train_error, generalis
        ax.scatter(x, y, color=color)
        ax.annotate(f"{meta_param}", xy=(x, y))

    ax.set_xlabel(r"$E_{train}$")
    ax.set_ylabel(r"Generalisation Gap")
```

In [15]:
```
fig = plt.figure()
ax = fig.add_subplot(111)
n=10
```

```
# generalisation_gap_vs_e_train(ax, get_metrics_quadratic, [0.1, 1.0, 10,
generalisation_gap_vs_e_train(ax, lambda n, lam: get_metrics_quadratic(n,
generalisation_gap_vs_e_train(ax, lambda n, lam: get_metrics_quadratic(n,
generalisation_gap_vs_e_train(ax, lambda n, lam: get_metrics_quadratic(n,
generalisation_gap_vs_e_train(ax, get_metrics_tree, [5, 4, 3, 2, 1], n=n,
plt.title(f"Using n={n}")
plt.xlim([0, 2.5])
plt.legend()
plt.show()
```



In [16]:
```
fig = plt.figure()
ax = fig.add_subplot(111)
n=20
# generalisation_gap_vs_e_train(ax, get_metrics_quadratic, [0.1, 1.0, 10,
generalisation_gap_vs_e_train(ax, lambda n, lam: get_metrics_quadratic(n,
generalisation_gap_vs_e_train(ax, lambda n, lam: get_metrics_quadratic(n,
generalisation_gap_vs_e_train(ax, lambda n, lam: get_metrics_quadratic(n,
generalisation_gap_vs_e_train(ax, get_metrics_tree, [5, 4, 3, 2, 1], n=n,
plt.title(f"Using n={n}")
plt.xlim([0, 2.5])
plt.legend()
plt.show()
```



I only producedthe green and blue curves as further evidence for Dr. Harrison that the value of n should be 20 not 10.

Problem 4

HW5:

Problem 4:

$$\bar{E}_{new} = \mathbb{E}[E_{new}(\mathcal{T})] = \mathbb{E}_{\mathcal{T}}\left[\mathbb{E}_{*}\left[\underbrace{(\hat{g}(\underline{X}_{*};\mathcal{T}) - y_{*})^2}_{\text{squared error function}}\right]\right]$$

$$= \mathbb{E}_{*}\left[\mathbb{E}_{\mathcal{T}}\left[\hat{g}(\underline{X}_{*};\mathcal{T}) - f_0(\underline{X}_{*}) - \epsilon\right)^2\right]] \quad (1)$$

recall ... $y = f_0(\underline{X}) + \epsilon$ with $\mathbb{E}[\epsilon] = 0$ $Var(\epsilon) = \sigma^2$

and $y_{*} = f_0(\underline{X}_{*}) + \epsilon$

$$\mathbb{E}_{\mathcal{T}}\left[(\hat{g}(\underline{X}_{*};\mathcal{T}) - f_0(\underline{X}_{*}) - \epsilon)^2\right]$$

$$= \mathbb{E}_{\mathcal{T}}\left[(\hat{g}(\underline{X}_{*};\mathcal{T}) \underbrace{- \bar{f}(\underline{X}_{*}) + \bar{f}(\underline{X}_{*})}_{\text{simply added 0, the additive identity}} - f_0(\underline{X}_{*}) - \epsilon)^2\right]$$

$$= \mathbb{E}_{\mathcal{T}}\left[\left(\left(\hat{g}(\underline{X}_{*};\mathcal{T}) - \bar{f}(\underline{X}_{*})\right) - \left(\bar{f}(\underline{X}_{*}) - \underbrace{f_0(\underline{X}_{*}) - \epsilon}_{-y_{*}}\right)\right)^2\right]$$

$$= \mathbb{E}_{\mathcal{T}}\left[\left(\left(\hat{g}(\underline{X}_{*};\mathcal{T}) - \bar{f}(\underline{X}_{*})\right) - \left(\bar{f}(\underline{X}_{*}) - y_{*}\right)\right)^2\right]$$

let $A = \hat{g}(\underline{X}_{*};\mathcal{T}) - \bar{f}(\underline{X}_{*})$, $B = \bar{f}(\underline{X}_{*}) - y_{*}$

$$\Rightarrow \mathbb{E}_{\mathcal{T}}\left[(\hat{g}(\underline{X}_{*};\mathcal{T}) - f_0(\underline{X}_{*}) - \epsilon)^2\right]$$

$$= \mathbb{E}_{\mathcal{T}}[(A - B)^2]$$

$$= \mathbb{E}_{\mathcal{T}}[A^2 - 2AB + B^2] \quad (2)$$

by the linearity of the expectation value operator, we rewrite eq (2)

$$\mathbb{E}_T\left[(\hat{y}(\underline{x}_*;T) - f_0(\underline{x}_*) - \epsilon)^2\right]$$

$$= \mathbb{E}_T\left[A^2 - 2AB + B^2\right]$$

$$= \mathbb{E}_T\left[A^2\right] - \mathbb{E}_T\left[2AB\right] + \mathbb{E}_T\left[B^2\right] \quad (3)$$

Plugging in for A & B ($\bar{f}(\underline{x}_*) - y_*$ is constant)

$$\mathbb{E}_T\left[2AB\right] = \mathbb{E}_T\left[2\cdot(\hat{y}(\underline{x}_*;T) - \bar{f}(\underline{x}_*))(\bar{f}(\underline{x}_*) - y_*)\right]$$

$$= 2\cdot\left[\mathbb{E}_T\left[\hat{y}(\underline{x}_*;T) - \bar{f}(\underline{x}_*)\right](\bar{f}(\underline{x}_*) - y_*)\right]$$

($\bar{f}(\underline{x}_*)$ constant)

$$= 2\left(\left[\mathbb{E}_T\left[\hat{y}(\underline{x}_*;T)\right] - \bar{f}(\underline{x}_*)\right)(\bar{f}(\underline{x}_*) - y_*)\right) \quad (4)$$

Recall, $\bar{f}(\underline{x}) \triangleq \mathbb{E}_T\left[\hat{y}(\underline{x}_*;T)\right]$

plugging into equation (4)

$$\mathbb{E}_T\left[2AB\right] = 2\left(\bar{f}(\underline{x}_*) - \bar{f}(\underline{x}_*)\right)(\bar{f}(\underline{x}_*) - y_*)$$

$$= 2\cdot 0\cdot(\bar{f}(\underline{x}_*) - y_*)$$

$$= 0$$

plugging into eq (3)

$$\mathbb{E}_T\left[(\hat{y}(\underline{x}_*;T) - f_0(\underline{x}_*) - \epsilon)^2\right] = \mathbb{E}_T\left[A^2\right] + \mathbb{E}_T\left[B^2\right]$$

Plugging in for A & B:

$$\mathbb{E}_\tau[(\hat{y}(\underline{x}_*;\tau) - f_0(\underline{x}_*) - \epsilon)^2]$$

$$= \mathbb{E}_\tau\left[(\hat{y}(\underline{x}_*;\tau) - \bar{f}(\underline{x}_*))^2\right] + \mathbb{E}_\tau\left[(\bar{f}(\underline{x}_*) - y_*)^2\right]$$

This term is not parameterized by τ

$$= \mathbb{E}_\tau\left[(\hat{y}(\underline{x}_*;\tau) - \bar{f}(\underline{x}_*))^2\right] + (\bar{f}(\underline{x}_*) - y_*)^2$$

$$= (\bar{f}(\underline{x}_*) - y_*)^2 + \mathbb{E}_\tau\left[(\hat{y}(\underline{x}_*;\tau) - \bar{f}(\underline{x}_*))^2\right]$$

$$= \bar{f}^2(\underline{x}_*) - 2y_*\bar{f}(\underline{x}_*) + y_*^2 + \mathbb{E}_\tau\left[(\hat{y}(\underline{x}_*;\tau) - \bar{f}(\underline{x}_*))^2\right]$$

$$= \bar{f}^2(\underline{x}_*) - 2(f_0(\underline{x}_*) + \epsilon)\bar{f}(\underline{x}_*) + (f_0(\underline{x}_*) + \epsilon)^2 + \mathbb{E}_\tau\left[(\hat{y}(\underline{x}_*;\tau) - \bar{f}(\underline{x}_*))^2\right]$$

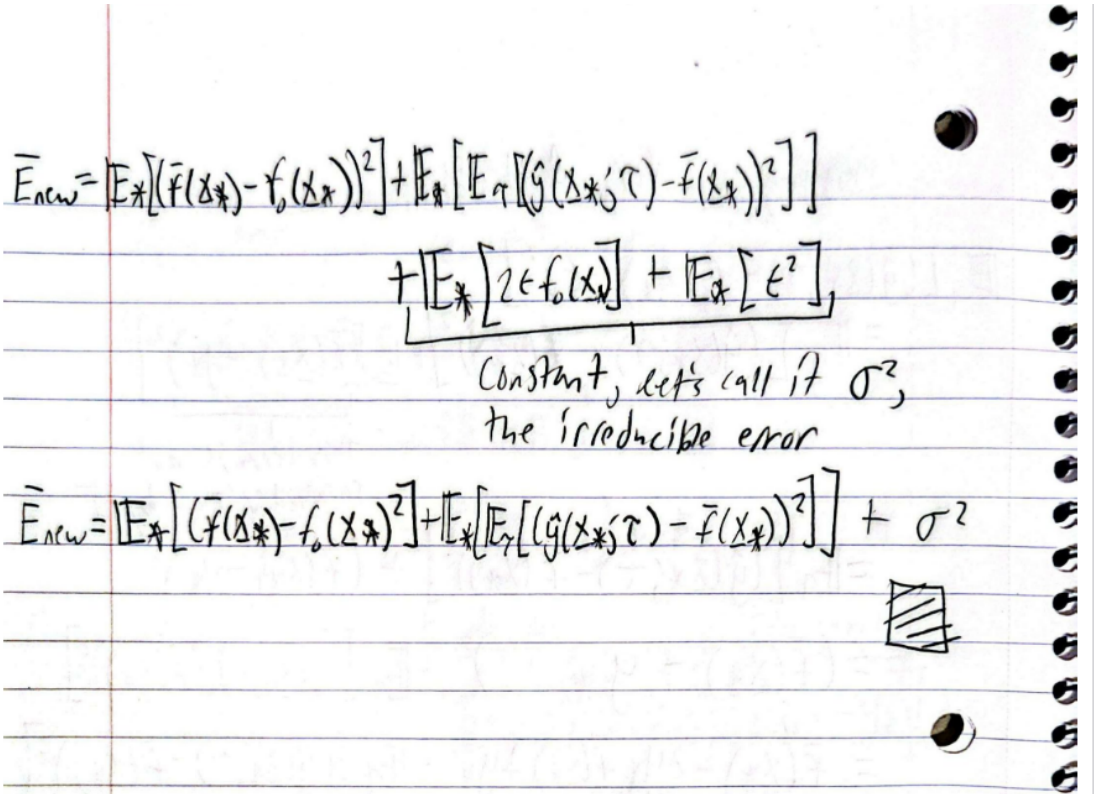
$$= \bar{f}^2(\underline{x}_*) - 2f_0(\underline{x}_*)\bar{f}(\underline{x}_*) + f_0^2(\underline{x}_*) - 2\epsilon f_0(\underline{x}_*) + \epsilon^2 + \mathbb{E}_\tau\left[(\hat{y}(\underline{x}_*;\tau) - \bar{f}(\underline{x}_*))^2\right]$$

$$= (\bar{f}(\underline{x}_*) - f_0(\underline{x}_*))^2 + \mathbb{E}_\tau\left[(\hat{y}(\underline{x}_*;\tau) - \bar{f}(\underline{x}_*))^2\right] - 2\epsilon f_0(\underline{x}_*) + \epsilon^2$$

Plugging into eq (1)

$$\bar{E}_{new} = \mathbb{E}_*\left[(\bar{f}(\underline{x}_*) - f_0(\underline{x}_*))^2 + \mathbb{E}_\tau\left[(\hat{y}(\underline{x}_*;\tau) - \bar{f}(\underline{x}_*))^2\right] + 2\epsilon f_0(\underline{x}_*) + \epsilon^2\right]$$

$$= \mathbb{E}_*\left[(\bar{f}(\underline{x}_*) - f_0(\underline{x}_*))^2\right] + \mathbb{E}_*\left[\mathbb{E}_\tau\left[(\hat{y}(\underline{x}_*;\tau) - \bar{f}(\underline{x}_*))^2\right]\right] + \mathbb{E}_*\left[2\epsilon f_0(\underline{x}_*) + \epsilon^2\right]$$



Problem 5

Precision and recall defined on page 86 Examples of ROC and precision recall curves on page 88

(a)

In [17]:
```
def calculate_x(n=1000):
    return np.random.choice([1, -1], size=n, p=[0.7, 0.3])

@np.vectorize
def _calculate_y_from_z(z, r):
    if z > r:
        return 1
    else:
        return -1

def calculate_y(x, r=0):
    n = x.shape[0]
    z = x + np.random.normal(0, 1.0, n)
    y = _calculate_y_from_z(z, r)
    return y

def calculate_confusion_matrix(r=0, n=1000):
    r = np.atleast_1d(r)
    confusion_matrix = {
        "TN": np.zeros_like(r),
        "FN": np.zeros_like(r),
        "FP": np.zeros_like(r),
        "TP": np.zeros_like(r),
        "N": np.zeros_like(r),
        "P": np.zeros_like(r),
```

```
        "N*": np.zeros_like(r),
        "P*": np.zeros_like(r),
        "n": np.zeros_like(r),
    }
    x = calculate_x(n=n)

    for i, _r in enumerate(r):
        y = calculate_y(x, _r)
        for xi, yi in zip(x, y):
            if xi == -1 and yi==-1:
                confusion_matrix["TN"][i] += 1
            elif xi == -1 and yi==1:
                confusion_matrix["FP"][i] += 1
            elif xi == 1 and yi==-1:
                confusion_matrix["FN"][i] += 1
            elif xi == 1 and yi==1:
                confusion_matrix["TP"][i] += 1
            else:
                pass

        confusion_matrix["N"][i] = confusion_matrix["TN"][i] + confusion_
        confusion_matrix["P"][i] = confusion_matrix["FN"][i] + confusion_
        confusion_matrix["N*"][i] = confusion_matrix["TN"][i] + confusion
        confusion_matrix["P*"][i] = confusion_matrix["FP"][i] + confusion
        confusion_matrix["n"][i] = len(x)
    return confusion_matrix
```

(a) What is the confusion matrix if r = 0? Also calculate the probability of false alarm, probability of missed detection, precision, and recall.

Below are the values for the confusion matrix

In [18]:
```
conf_mat = calculate_confusion_matrix(r=0, n=1000)
```

In [19]:
```
print(f"       " + "\t  \t" + "y=-1" + "\t" + "y=1" + "\t" + "total")
print("y_hat=-1" + "\t" + f"{conf_mat["TN"]}" + "\t" + f"{conf_mat["FN"]}
print("y_hat=1" + "\t \t" + f"{conf_mat["FP"]}" + "\t" + f"{conf_mat["TP"
print("total" + "\t \t" + f"{conf_mat["N"]}" + "\t" + f"{conf_mat["P"]}"
```

```
                y=-1    y=1     total
y_hat=-1        [256]   [112]   [368]
y_hat=1         [50]    [582]   [632]
total           [306]   [694]   [1000]
```

In [20]:
```
conf_mat = calculate_confusion_matrix(r=0, n=1000)
p_false_alarm = conf_mat["FP"] / (conf_mat["N"])
p_missed_detection = conf_mat["FN"] / (conf_mat["P"])
precision = conf_mat["TP"] / (conf_mat["P*"])
recall = conf_mat["TP"] / conf_mat["P"]

print(f"probability of false alarm (r=0): {p_false_alarm}")
print(f"probability of missed detection (r=0): {p_missed_detection}")
print(f"recall (r=0): {recall}")
print(f"precision (r=0): {precision}")
```

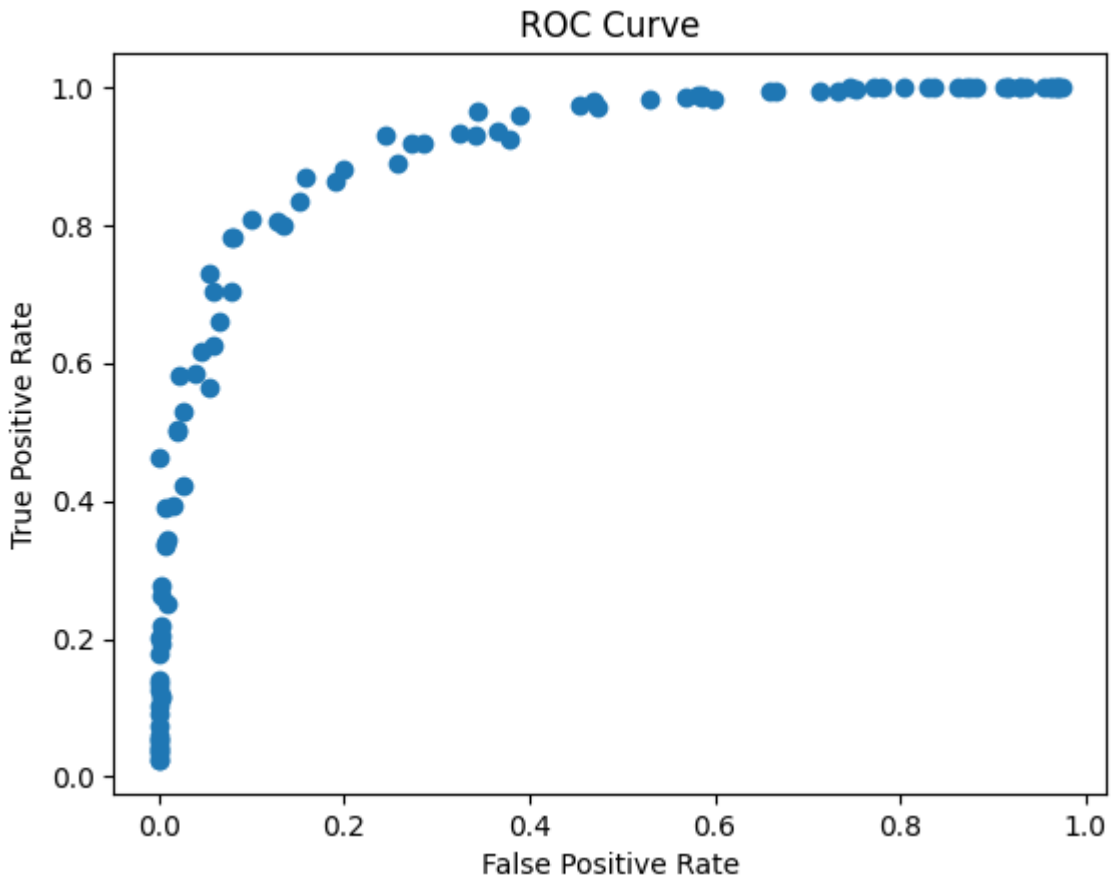
```
probability of false alarm (r=0): [0.14851485]
probability of missed detection (r=0): [0.14634146]
recall (r=0): [0.85365854]
precision (r=0): [0.9296875]
```

(b)

In [21]:
```
def calculate_false_positive_rate(confusion_matrix):
    return confusion_matrix["FP"] / (confusion_matrix["N"])

def calculate_true_positive_rate(confusion_matrix):
    return confusion_matrix["TP"] / (confusion_matrix["P"])
```

In [22]:
```
# Calculate the ROC curve (true positive rate vs false positive rate)
r = np.linspace(-3, 3, 100)
conf_mat = calculate_confusion_matrix(r=r, n=1000)
tp_rate = calculate_true_positive_rate(conf_mat)
fp_rate = calculate_false_positive_rate(conf_mat)

plt.scatter(fp_rate, tp_rate)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
```

Out[22]: Text(0.5, 1.0, 'ROC Curve')



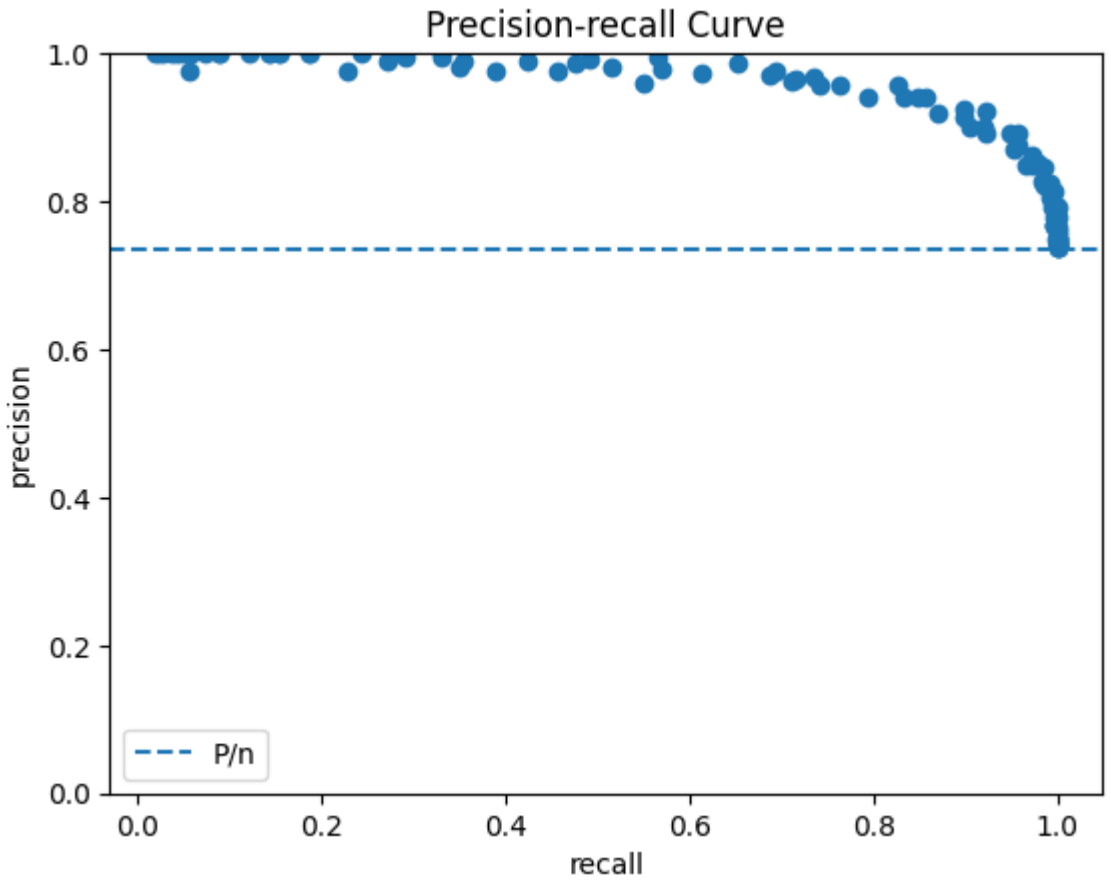
(c)

In [23]:
```
def calculate_recall(confusion_matrix):
    return confusion_matrix["TP"] / (confusion_matrix["P"])

def calculate_precision(confusion_matrix):
    return confusion_matrix["TP"] / (confusion_matrix["P*"])
```

In [24]:
```
r = np.linspace(-3, 3, 100)
conf_mat = calculate_confusion_matrix(r=r, n=1000)
```

```
recall = calculate_recall(conf_mat)
precision = calculate_precision(conf_mat)

plt.scatter(recall, precision)
plt.axhline((conf_mat["P"]/conf_mat["n"])[0], linestyle="--", label="P/n"
plt.xlabel("recall")
plt.ylabel("precision")
plt.title("Precision-recall Curve")
plt.ylim([0.0, 1.0])
plt.legend()
```

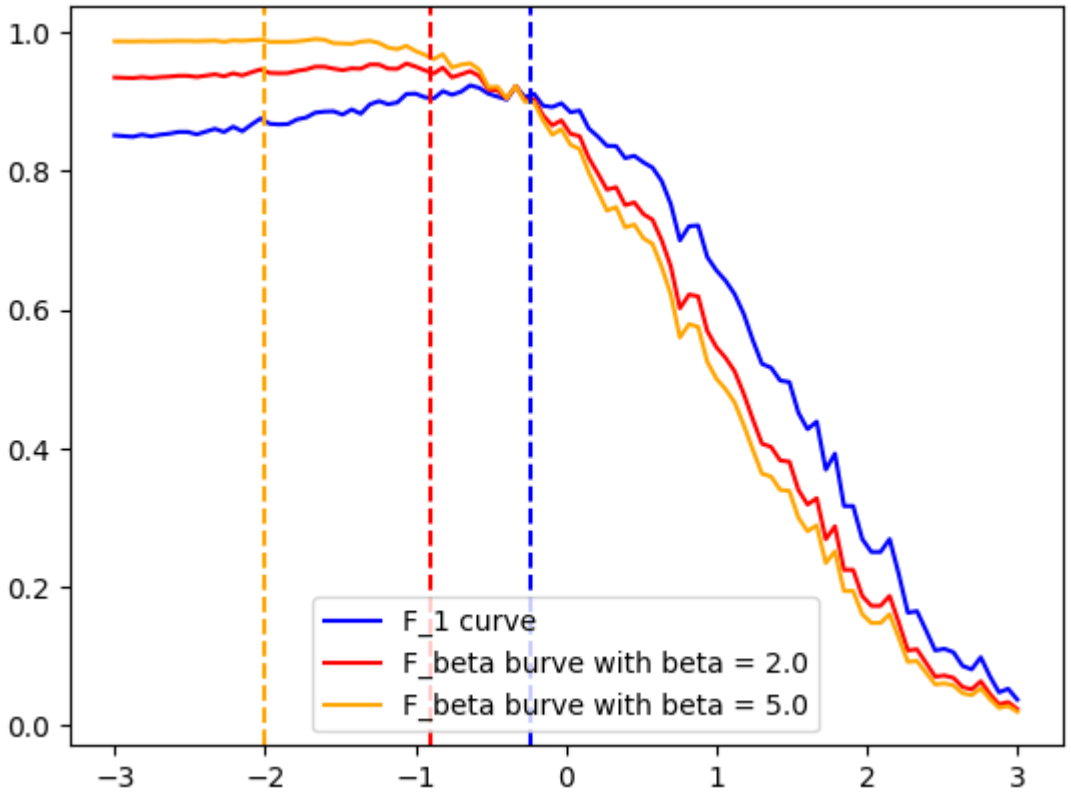
Out[24]: <matplotlib.legend.Legend at 0x7c45216751c0>



(d) What values of r give the best predictors according to each curve? Are they the same? Why or why not?

In [25]:
```
F_1 = 2*precision*recall/(precision+recall)


plt.plot(r, F_1, label="F_1 curve", color="blue")
beta = 2.0
F_beta = (1+beta**2)*precision*recall/(beta**2 * precision+recall)
plt.plot(r, F_beta, label=f"F_beta burve with beta = {beta}", color="red"

beta = 5.0
F_beta = (1+beta**2)*precision*recall/(beta**2 * precision+recall)
plt.plot(r, F_beta, label=f"F_beta burve with beta = {beta}", color="oran

plt.axvline(-0.23, color="blue", linestyle="--")
plt.axvline(-0.9, color="red", linestyle="--")
plt.axvline(-2.0, color="orange", linestyle="--")
plt.legend()
```

Out[25]: <matplotlib.legend.Legend at 0x7c45214df560>



plt.plot

The question of which r value is best really depends on the context of what we are modelling and whether a false positive of a false negative is more damaging. There are some metrics which we can use, however. The F_1 curve is maximized at the value r=0.23, which corresponds to the r value that caused the true positive rate to be approximately 78% and the false positive rate to be approximately 10%. Without further knowledge of the scenario that we are modelling the F_1 prediction of r=0.23 seems to strike a decent balance between maximizing the true positive rate and minimizing the false positive rate. It the false positive rate is unacceptable, then I would suggest increasing the r value. Increasing the r value to r=0.6 will bring the false positive rate to well below 5% and increasing. If false negatives are an issue (missed detection), then obviously you should lower the r value. Regardless these decisions are context dependent and I cannot provide one true answer. In the absence of the context of what we are modeling, I will go with the r value predicted by the F_1 curve:

r=0.23

This would correspond to the r-value that

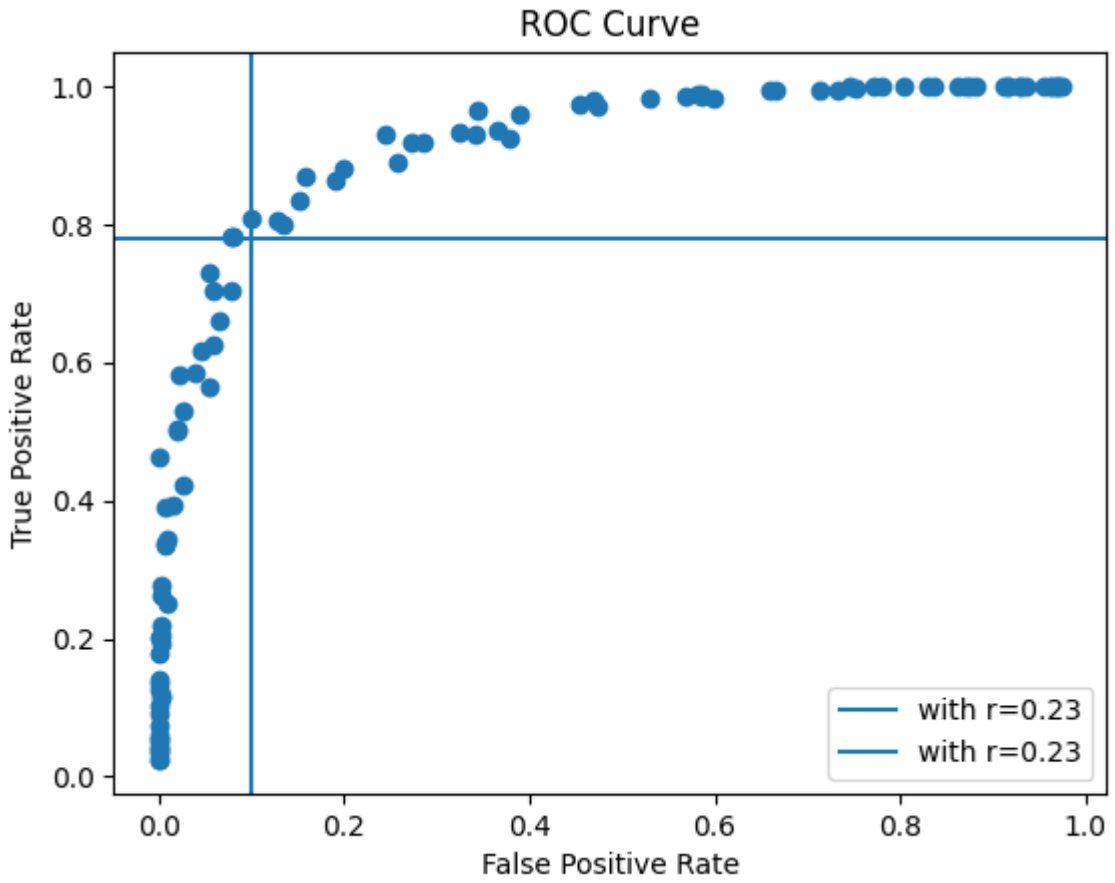
In [26]:
```
r = 0.23
conf_mat = calculate_confusion_matrix(r=r, n=1000)

print(calculate_true_positive_rate(conf_mat))
print(calculate_false_positive_rate(conf_mat))

print( calculate_recall(conf_mat))
```

```
print(calculate_precision(conf_mat))
```

```
[0.78450704]
[0.0862069]
[0.78450704]
[0.95704467]
```

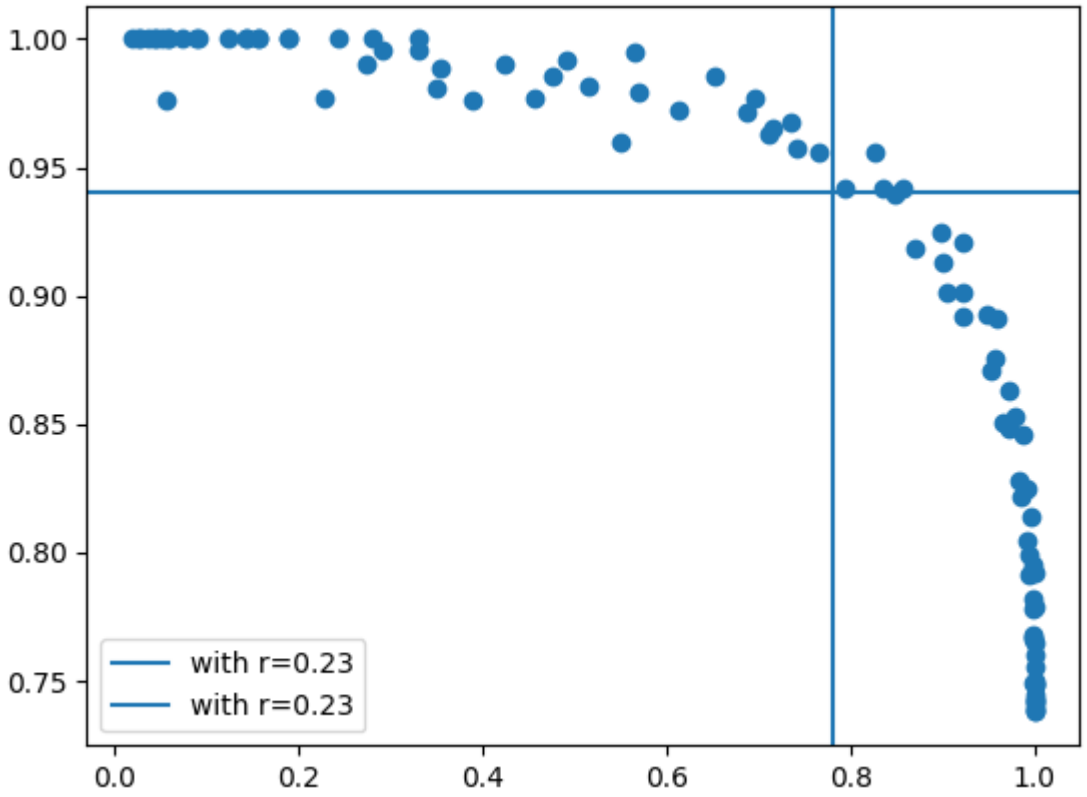
In [ ]:
```
# I determined the values of 0.10 and 0.78 by plugging in r=0.23 instead
plt.scatter(fp_rate, tp_rate)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.axvline(0.10, label="with r=0.23")
plt.axhline(0.78, label="with r=0.23")
plt.legend()
plt.title("ROC Curve")
```

Out[ ]:
```
Text(0.5, 1.0, 'ROC Curve')
```



In [ ]:
```
# I determined the values of 0.78 and 0.94 by plugging in r=0.23 instead
plt.scatter(recall, precision)
plt.axvline(0.78, label="with r=0.23")
plt.axhline(0.94, label="with r=0.23")
plt.legend()
```

Out[ ]:
```
<matplotlib.legend.Legend at 0x7c4521466ea0>
```



Clearly our choice of r=0.23 strikes the point of greatest curvature on the ROC curve and is near the point of greatest curvature on the precision-recall curve. This suggests that both curves agree (assuming the point of greatest curvature strikes a good balance between high precision, high recall, high true positive rate, and low false positive rates) that r=0.23 is a decent value when all these metrics are equally important in determining the optimacy of the r value.